Математическое моделирование

Лабораторная работа №4
«Системы массового обслуживания»

Спецификация программы: теория, решение задач, код

Содержание

1 Обзор 3
1.1 Что делает программа 3
1.2 Стек и запуск 3
1.3 Модули 3
1.4 Поток данных 4
2 Теория 5
2.1 Обозначения 5
2.2 Система массового обслуживания 5
2.3 Поток событий. Простейший поток 5
2.4 Показательное время и отсутствие памяти 6
2.5 Состояния и граф состояний 6
2.6 Финальные вероятности 6
2.7 Метод сечений для цепочек 7
2.8 Условие существования стационарного режима 7
2.9 Характеристики эффективности 7
3 Как программа решает задачу 8
3.1 Решатель: `chain.rs` 8
3.2 Описание задачи: `task.rs` 8
3.3 Интерфейс: `task_tab.rs` 9
3.4 Бесконечная очередь 9
4 Задачи 1 и 3: n каналов, m мест в очереди 10
4.1 Граф и формулы 10
4.2 Задача 1: линия связи с отказами 10
4.3 Задача 3: сортировочная горка с неограниченной очередью 11
5 Станки и оператор 13
5.1 Граф 13
5.2 Решение 13
6 Одноканальная СМО с отказами канала 15
6.1 Состояния 15
6.2 Переходы 15
6.3 Решение 16
6.4 Характеристики 16
7 Приоритет с вытеснением 17
7.1 Состояния и переходы 17
7.2 Решение 17
7.3 Характеристики 18
8 Метод фаз: обслуживание по закону Эрланга 19
8.1 Идея метода 19
8.2 Состояния и переходы 19
8.3 Задача без очереди и «нечувствительность» 20
8.4 Характеристики из вектора p 20
8.5 Решение задачи про ЭВМ 20
9 Магазин: три СМО подряд 22
9.1 Почему звенья считаются по отдельности 22
9.2 Формулы Эрланга C 22

9.3 Решение 22
9.4 Ответ на дополнительный вопрос 23
10 Остальные немарковские задачи (не реализованы) 24
11 Тесты 25
12 Допущения и ограничения 27

1 Обзор

1.1 Что делает программа

В `ЛР4задания.docx` собраны около тридцати задач по теории массового обслуживания (СМО): системы с отказами, с очередью, с «нетерпеливыми» заявками, с отказами канала, с приоритетами, с взаимопомощью каналов и так далее. Почти все они устроены одинаково: нужно нарисовать граф состояний, записать уравнения для финальных вероятностей, решить их и выразить через вероятности характеристики системы.

Программа `lab4` решает **четыре задачи** такого типа (они выбраны так, чтобы покрыть разные виды графа) и ещё две задачи, которые в эту схему не укладываются: обслуживание по закону Эрланга (метод фаз) и магазин из трёх последовательных СМО.

Вкладка	**Задача**	**Чем интересна**
Channels and queue	n каналов, m мест в очереди или неограниченная очередь; покрывает задачи 1, 3, АЗС, ограниченную очередь	Цепочка состояний, одно число k — число заявок в системе
Machines and repairman	Оператор налаживает n станков	Интенсивность «вверх» зависит от состояния: $(n-k)\lambda$
Channel failures	Одноканальная СМО с очередью $m = 2$, канал выходит из строя и ремонтируется	Состояния двух видов: «канал исправен» и «канал ремонтируется»
Priority with preemption	Двухканальная СМО с отказами и двумя потоками, заявки I вытесняют заявки II	Двумерные состояния (r, j) и переход-вытеснение
Phase-type service	Одноканальная СМО, время обслуживания распределено по закону Эрланга или обобщённому Эрлангу; задача про ЭВМ	Немарковская задача сводится к марковской добавлением фаз обслуживания (раздел 8)
Shop (series of queues)	Магазин: выбор товара, касса, контроль, три СМО подряд	Нет графа всей системы: звенья считаются по формулам Эрланга С (раздел 9)

Остальные немарковские задачи (фиксированное время обслуживания, столовая, две фазы с возвратом) в программу **не входят**. Их можно решить формулами; краткая классификация дана в разделе 10.

1.2 Стек и запуск

Rust (edition 2024), `eframe`/`egui` и `egui_plot` для окна и графиков, `nalgebra` для решения линейной системы. Крейт `lab4` входит в cargo-workspace корня репозитория и использует `common` (трейт `Tab` и цвета), как `lab2`.

```
cargo run -p lab4          # окно с шестью вкладками
cargo test -p lab4         # 24 теста
cargo clippy -p lab4 --all-targets
```

1.3 Модули

Файл	**Роль**
`chain.rs`	Решатель. `Chain` хранит интенсивности переходов, `stationary()` возвращает финальные вероятности.

`task.rs`	Описание задачи для интерфейса: `Param` (параметр), `Task` (имя, параметры, функция решения), `Solution` (подписи состояний, вероятности, характеристики, предупреждения).
`tasks/queue.rs`	Семейство «n каналов, m мест в очереди или неограниченная очередь».
`tasks/machines.rs`	Станки и оператор.
`tasks/breakdown.rs`	Канал с отказами, очередь m мест.
`tasks/priority.rs`	Два потока заявок, приоритет с вытеснением.
`tasks/phases.rs`	Одноканальная СМО с фазовым (эрланговским) временем обслуживания.
`tasks/shop.rs`	Три последовательные многоканальные СМО, формулы Эрланга С.
`task_tab.rs`	Одна вкладка для любой `Task`: поля параметров, диаграмма вероятностей, таблицы.
`main.rs`	Окно с верхней панелью вкладок (как в `lab2`).

1.4 Поток данных

1. Вкладка показывает поля параметров по списку `Task::params`.
2. При любом изменении поля собирается вектор значений и вызывается `Task::solve`.
3. Функция задачи проверяет параметры, строит граф (`Chain::add_rate` для каждого перехода) и вызывает `stationary()`.
4. Из полученных вероятностей p функция сама считает характеристики и возвращает `Solution`.
5. Вкладка рисует столбцы p_k по подписям состояний и две таблицы: вероятности и характеристики (для сетевой задачи «Shop» распределения p нет, поэтому показывается только таблица характеристик). Если `solve` вернула ошибку (например, нет стационарного режима), вместо результата показывается красное сообщение.

> Решатель ничего не знает про конкретные задачи. Он принимает только граф. Смысл состояний знает только сама задача, поэтому именно она выводит $P_{\text{отк}}$, A, длину очереди и остальное из вектора p.

2 Теория

2.1 Обозначения

λ	интенсивность потока заявок: среднее число заявок в единицу времени
μ	интенсивность обслуживания одним каналом, $\mu = \frac{1}{t_{\text{обсл}}}$
n	число каналов
m	число мест в очереди
k	число заявок в системе (обслуживаемых и ожидающих)
p_k	финальная вероятность состояния k
$\alpha = \frac{\lambda}{\mu}$	приведённая интенсивность (нагрузка), число «занятых каналов» при бесконечно быстром потоке
$\rho = \frac{\alpha}{n}$	загрузка одного канала
$P_{\text{отк}}$	вероятность отказа
Q	относительная пропускная способность: доля обслуженных заявок
A	абсолютная пропускная способность: сколько заявок обслуживается в единицу времени
z	среднее число заявок в системе
$L_{\text{оч}}$	среднее число заявок в очереди
$t_{\text{сист}}, t_{\text{оч}}$	среднее время пребывания заявки в системе и в очереди
$\overline{k}$	среднее число занятых каналов

2.2 Система массового обслуживания

СМО — любая система, в которую приходят заявки и в которой есть каналы обслуживания. Примеры: линия телефонной связи, кассир, сортировочная горка, заправочная колонка, станок, который чинит оператор. Заявки приходят в случайные моменты, и время обслуживания тоже случайное, поэтому ответ на вопрос «как хорошо справляется система» — это **средние** величины за долгое время.

Системы различают по тому, что происходит с заявкой, заставшей все каналы занятыми:

- **с отказами**: заявка уходит необслуженной ($m = 0$);
- **с очередью**: заявка встаёт в очередь; очередь может быть **ограниченной** (m мест; если очередь полна, отказ) или **неограниченной**;
- **с нетерпеливыми заявками**: ожидающая заявка может уйти, не дождавшись (в программе не реализовано).

2.3 Поток событий. Простейший поток

Поток событий — последовательность однотипных событий в случайные моменты времени (приход заявок, завершение обслуживания, отказ канала). Главная характеристика — **интенсивность** λ, среднее число событий в единицу времени. Если $\lambda = 2$ в час, то среднее время между событиями равно $\frac{1}{\lambda} = 0.5$ часа.

Простейший (пуассоновский) поток — поток с тремя свойствами:

- **стационарность**: λ не меняется со временем;
- **отсутствие последействия**: число событий в одном промежутке не влияет на число событий в другом, не пересекающемся с ним;
- **ординарность**: два события не происходят одновременно.

Из этих свойств следует, что число событий за время t имеет распределение Пуассона со средним λt, а **интервал между соседними событиями распределён показательно**:

$$\mathrm{P}(T < t) = 1 - e^{-\lambda t}, \quad t \geq 0, \quad M(T) = \frac{1}{\lambda}.$$

Это то самое показательное распределение, которое имитировалось в лабораторной №2.

2.4 Показательное время и отсутствие памяти

Показательное распределение обладает свойством «отсутствия памяти»: если событие ещё не наступило к моменту s, то оставшееся время ожидания распределено так же, как и в момент 0:

$$\mathrm{P}(T > s + t \mid T > s) = \frac{e^{-\lambda(s+t)}}{e^{-\lambda s}} = e^{-\lambda t} = \mathrm{P}(T > t).$$

Следствие для СМО: если время обслуживания показательное с параметром μ, то вероятность закончить обслуживание в ближайшую малую долю времени $\mathrm{d}t$ равна $\mu\,\mathrm{d}t$ **независимо от того, как давно оно идёт**. Поэтому для описания системы достаточно знать, в каком она состоянии, а предыстория не нужна.

> Условия «поток простейший» и «время обслуживания показательное» в задачах нужны именно ради этого свойства. Если время обслуживания, например, строго равно 3 минутам (задача 2) или распределено по Эрлангу (задачи про ЭВМ), то система перестаёт быть марковской и граф с постоянными интенсивностями не подходит.

2.5 Состояния и граф состояний

Состояние — минимальная информация о системе, по которой можно предсказать её будущее. В простейших задачах это число заявок в системе $k = 0, 1, 2, \ldots$. В более сложных состояние включает ещё и исправность канала или тип заявок. Систему с конечным или счётным числом состояний, в которой время пребывания в каждом состоянии показательное, называют **марковским процессом с непрерывным временем**.

Граф состояний — это кружки-состояния и стрелки-переходы. Над стрелкой $i \to j$ пишется **интенсивность перехода** λ_{ij}: среднее число переходов из i в j за единицу времени, пока система находится в i. Откуда берутся интенсивности:

- пришла заявка: $k \to k+1$ с интенсивностью λ;
- закончилось обслуживание: $k \to k-1$ с интенсивностью μ на каждый **занятый** канал, то есть $\min(k, n)\mu$ для n каналов;
- отказ канала, ремонт: с интенсивностями Z и R (по условию).

Если в состоянии i возможны несколько событий, система покидает его с суммарной интенсивностью $\sum_j \lambda_{ij}$, а выбирает переход пропорционально интенсивностям.

2.6 Финальные вероятности

Пусть $p_i(t)$ — вероятность быть в состоянии i в момент t. За малое время $\mathrm{d}t$ в состояние i входят потоки из соседей, а выходят потоки в соседей, поэтому

$$\frac{\mathrm{d}p_i}{\mathrm{d}t} = \sum_j \lambda_{ji} p_j - p_i \sum_j \lambda_{ij} \quad \text{(уравнения Колмогорова)}.$$

При $t \to \infty$ для большинства систем вероятности перестают меняться: $p_i(t) \to p_i$. Эти предельные значения называются **финальными** (стационарными) вероятностями. Их можно понимать как долю времени, которую система проводит в состоянии i. Приравняв производные к нулю, получаем **уравнения баланса**:

$$p_i \sum_j \lambda_{ij} = \sum_j \lambda_{ji} p_j, \quad \sum_i p_i = 1.$$

«Поток из состояния равен потоку в состояние». Это однородная линейная система, в которой одно уравнение лишнее (сумма всех уравнений тождественно равна нулю), поэтому её дополняют условием нормировки $\sum p_i = 1$.

В матричном виде: пусть Q — матрица, у которой вне диагонали стоят интенсивности $q_{ij} = \lambda_{ij}$, а на диагонали $q_{ii} = -\sum_{j \neq i} \lambda_{ij}$. Тогда уравнения баланса — это

$$pQ = 0, \quad p \cdot \mathbf{1} = 1.$$

Именно эту систему решает программа.

Пример: задача 1. Два состояния: 0 (канал свободен), 1 (занят), $0 \to 1$ с интенсивностью λ, $1 \to 0$ с интенсивностью μ.

$$Q = \begin{pmatrix} -\lambda & \lambda \\ \mu & -\mu \end{pmatrix}, \quad \begin{cases} \lambda p_0 = \mu p_1 \\ p_0 + p_1 = 1 \end{cases} \implies p_0 = \frac{\mu}{\lambda + \mu}, \quad p_1 = \frac{\lambda}{\lambda + \mu}.$$

2.7 Метод сечений для цепочек

Если граф — цепочка $0 \leftrightarrow 1 \leftrightarrow 2 \leftrightarrow \ldots$, то уравнения баланса удобнее записывать для **сечения** между k и $k+1$. Всё, что перешло вправо через сечение, должно вернуться влево:

$$\lambda_k p_k = \mu_{k+1} p_{k+1} \implies p_{k+1} = \frac{\lambda_k}{\mu_{k+1}} p_k,$$

где λ_k — интенсивность $k \to k+1$, а μ_{k+1} — интенсивность $k+1 \to k$. Тогда

$$p_k = p_0 \cdot \prod_{i=0}^{k-1} \frac{\lambda_i}{\mu_{i+1}}, \quad p_0 = \left(\sum_k \prod_{i=0}^{k-1} \frac{\lambda_i}{\mu_{i+1}} \right)^{-1}.$$

Для сложных графов (отказы каналов, двумерные состояния) метод сечений не работает, и приходится решать систему целиком.

2.8 Условие существования стационарного режима

Для конечного графа, из любого состояния которого можно попасть в любое другое, финальные вероятности существуют и единственны. Для **неограниченной** очереди сумма $\sum p_k$ должна сойтись. В случае n каналов с одним потоком условие такое:

$$\rho = \frac{\lambda}{n\mu} < 1.$$

Если $\lambda \geq n\mu$, заявки приходят быстрее, чем канал(ы) могут обслуживать, очередь растёт без предела, и финальных вероятностей нет. Программа в этом случае не считает, а сообщает об отсутствии режима.

2.9 Характеристики эффективности

Все характеристики — это средние по финальным вероятностям.

- **Вероятность отказа** $P_{\text{отк}}$ — вероятность того, что приходящая заявка получит отказ. Простейший поток обладает свойством PASTA: заявки видят систему в той доле времени, в которой она в данном состоянии, поэтому $P_{\text{отк}}$ равна суммарной вероятности «блокирующих» состояний (всех каналов заняты и очередь полна).
- **Относительная пропускная способность** $Q = 1 - P_{\text{отк}}$ — доля обслуженных заявок.
- **Абсолютная пропускная способность** $A = \lambda Q$ — сколько заявок в единицу времени обслуживается.
- **Среднее число занятых каналов** $\overline{k} = \sum_k \min(k, n) p_k$. Каждый занятый канал завершает μ обслуживаний в единицу времени, поэтому $A = \mu \overline{k}$.
- **Среднее число заявок в системе** $z = \sum_k k p_k$ и **в очереди** $L_{\text{оч}} = \sum_{k>n} (k-n) p_k$.
- **Формула Литтла**: среднее число заявок в системе равно произведению интенсивности **принимаемого** потока на среднее время пребывания: $z = \lambda_{\text{прин}} t_{\text{сист}}$. Так же для очереди: $L_{\text{оч}} = \lambda_{\text{прин}} t_{\text{оч}}$. Отсюда $t_{\text{сист}} = \frac{z}{\lambda_{\text{прин}}}$ и $t_{\text{оч}} = \frac{L_{\text{оч}}}{\lambda_{\text{прин}}}$. Здесь $\lambda_{\text{прин}} = \lambda(1 - P_{\text{отк}})$ — только те заявки, которые не получили отказ. Формулу Литтла часто применяют к λ вместо $\lambda_{\text{прин}}$, и это ошибка: отказавшие заявки в системе не пребывают.

3 Как программа решает задачу

3.1 Решатель: `chain.rs`

`Chain` хранит квадратную матрицу интенсивностей `rates`: `rates[(i, j)]` — интенсивность перехода $i \to j$.

- `Chain::new(n)` — матрица $n \times n$ из нулей.
- `add_rate(from, to, rate)` — **добавляет** интенсивность в ячейку (`+=`). Это важно для случая, когда два разных события ведут из одного состояния в одно и то же.
- `stationary()` — собирает Q, решает систему и возвращает вероятности или `None`.

Шаги `stationary()`:

1. Скопировать `rates` в `q` и на диагональ поставить минус сумму строки: `q[(i, i)] = -rates.row(i).sum()`.
2. Транспонировать: система $pQ = 0$ записывается как $Q^T p^T = 0$, то есть в обычном виде $Ax = b$ с матрицей $A = Q^T$.
3. Одно уравнение лишнее, поэтому **заменить последнюю строку** на единицы: `a.row_mut(n - 1).fill(1.0)`. Это условие нормировки $\sum p_i = 1$.
4. Правая часть $b = (0, \ldots, 0, 1)^T$.
5. Решить LU-разложением: `a.lu().solve(&b)`. Если матрица вырождена (у системы нет единственного решения), вернётся `None`.
6. Проверить, что все числа конечны.

Вызов	Что делает
`DMatrix::zeros(n, m)`	Динамическая матрица $n \times m$ из нулей (размер известен только во время работы программы).
`m[(i, j)]`	Элемент строки i и столбца j; индексация кортежем.
`m.row(i).sum()`	Сумма элементов i-й строки.
`m.transpose()`	Транспонированная матрица (новая).
`m.row_mut(i).fill(x)`	Записывает x во все элементы i-й строки на месте.
`DVector::zeros(n)`	Вектор-столбец из n нулей.
`a.lu()`	LU-разложение с выбором ведущего элемента.
`lu.solve(&b)`	Решение $Ax = b$; `Option` — `None`, если A вырождена.
`v.as_slice().to_vec()`	Копия данных вектора в обычный `Vec<f64>`.

Почему не формула, а матрица: для цепочек вроде задачи 3 формулы $p_k = (1 - \rho)\rho^k$ достаточно, но для задачи с отказами канала или с приоритетами формулы нет. Матричный подход одинаково работает для всех задач, а для проверки правильности можно сравнивать с формулами там, где они есть (см. раздел про тесты).

3.2 Описание задачи: `task.rs`

`Task` состоит из названия, списка параметров `Vec<Param>` и функции `solve: fn(&[f64]) -> Result<Solution, String>`. Параметры бывают трёх видов (`Kind`): вещественное число, целое число со своим минимумом (`Count`) и флаг (`Flag`; в `Task` хранится как 0 или 1). Внутри `solve` значения читаются одним образцом:

```
let &[lambda, mu, n, m, unbounded] = params else { return Err(...) };
```

Это **slice pattern**: срез из ровно пяти чисел разбирается сразу на пять переменных.

`Solution` содержит подписи состояний (`labels`), вероятности (`p`), характеристики (`metrics`, пары «название — значение») и предупреждения (`warnings`). Методы `metric(...)` и `warning(...)` принимают и возвращают `self`, поэтому результат собирается цепочкой вызовов.

3.3 Интерфейс: `task_tab.rs`

Одна и та же структура `TaskTab` показывает любую задачу. `edit_param` рисует для вещественного параметра `DragValue` (число, которое можно перетаскивать или вписать), для целого — `DragValue` с минимумом, для флага — `checkbox`. Если значение изменилось, вызывается `recompute()`. Подписи состояний подставляются на ось x диаграммы через `x_axis_formatter`: по номеру столбца берётся `labels[i]`. Таблица состояний лежит в прокручиваемой области, поэтому при обрезании неограниченной очереди на 200 состояниях окно не растягивается.

3.4 Бесконечная очередь

В неограниченной очереди состояний бесконечно много, а матрица должна быть конечной. Поэтому граф обрезается на $N = 200$ состояниях. Остаток вероятности $\sum_{k>N} p_k = \rho^{N+1}$. Для $\rho = \frac{2}{3}$ это порядка 10^{-36} и обрезание ничего не меняет, но при $\rho = 0.99$ остаток равен $0.99^{201} \approx 0.13$, и результат неверен. Поэтому программа:

- не считает, если $\lambda \geq n\mu$ (стационарного режима нет);
- показывает красное предупреждение, если после решения $p_N > 10^{-9}$.

4 Задачи 1 и 3: n каналов, m мест в очереди

Вкладка **Channels and queue** имеет параметры λ, μ, n, m и флаг «unbounded queue». Состояние $k = 0, 1, \ldots, n+m$ — число заявок в системе. Для флага «unbounded» состояний 201 (от 0 до 200).

4.1 Граф и формулы

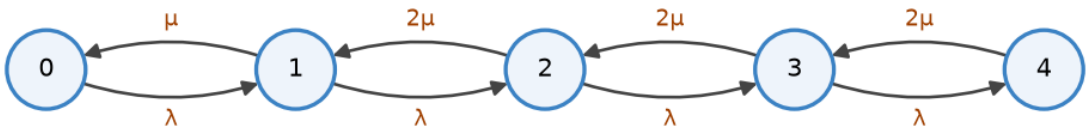


Рис. 1. Граф для $n = 2$, $m = 2$: интенсивность «вниз» растёт до $n\mu$, пока заняты не все каналы, потом остаётся равной $n\mu$.

Интенсивность $k \to k+1$ равна λ при любом $k < n+m$. Интенсивность $k \to k-1$ равна $\min(k, n)\mu$: занятых каналов столько, сколько заявок, но не больше n. Метод сечений ($\lambda p_k = \min(k+1, n)\mu p_{k+1}$) даёт

$$p_k = \begin{cases} \frac{\alpha^k}{k!}\, p_0 & \text{при } 0 \le k \le n \\ \frac{\alpha^k}{n!\, n^{k-n}}\, p_0 & \text{при } n < k \le n+m \end{cases}, \quad \alpha = \frac{\lambda}{\mu}.$$

Значение p_0 находится из нормировки. Предельные случаи:

- $m = 0$: **формула Эрланга** $P_{\text{отк}} = p_n = \frac{\frac{\alpha^n}{n!}}{\sum_{i=0}^{n} \frac{\alpha^i}{i!}}$;
- $n = 1$, $m = \infty$ и $\rho = \alpha < 1$: $p_k = (1-\rho)\rho^k$.

Характеристики (все через p, см. раздел с определениями):

$$P_{\text{отк}} = \begin{cases} p_{n+m} & \text{если очередь ограничена} \\ 0 & \text{если очередь неограничена} \end{cases}, \quad Q = 1 - P_{\text{отк}}, \quad A = \lambda Q,$$

$$z = \sum k p_k, \quad L_{\text{оч}} = \sum_{k>n} (k-n) p_k, \quad \overline{k} = \sum \min(k, n) p_k, \quad t_{\text{сист}} = \frac{z}{A}, \quad t_{\text{оч}} = \frac{L_{\text{оч}}}{A}.$$

В программе деление на A — это формула Литтла для принятого потока.

4.2 Задача 1: линия связи с отказами

Условие. Одноканальная СМО с отказами: $\lambda = 0.4$ вызова/мин, $t_{\text{обсл}} = 3$ мин. Найти p_0, p_1, A, Q, $P_{\text{отк}}$ и сравнить пропускную способность с номинальной.

Решение.

1. $\mu = \frac{1}{t_{\text{обсл}}} = \frac{1}{3}$ вызова/мин. Единицы λ и μ совпадают (вызовы в минуту).
2. Состояния: 0 — канал свободен, 1 — занят. Это $n = 1$, $m = 0$.
3. Уравнение баланса: $\lambda p_0 = \mu p_1$; нормировка $p_0 + p_1 = 1$. Тогда

$$p_0 = \frac{\mu}{\lambda + \mu} = \frac{\frac{1}{3}}{0.4 + \frac{1}{3}} = \frac{5}{11} \approx 0.4545, \quad p_1 = \frac{6}{11} \approx 0.5455.$$

4. $P_{\text{отк}} = p_1 = \frac{6}{11} \approx 0.5455$ — заявка отказывается, если канал занят.
5. $Q = 1 - P_{\text{отк}} = p_0 = \frac{5}{11} \approx 0.4545$.
6. $A = \lambda Q = 0.4 \cdot \frac{5}{11} = \frac{2}{11} \approx 0.1818$ вызова/мин.
7. **Сравнение с номиналом.** Номинальная пропускная способность — $\mu = \frac{1}{3}$ вызова/мин (канал обслуживает без простоев). Отношение $\frac{A}{\mu} = \frac{\frac{2}{11}}{\frac{1}{3}} = \frac{6}{11} \approx 0.55$: реальная система даёт 55 % от номинала. Это та же величина p_1, потому что $A = \mu\overline{k} = \mu p_1$: канал производит обслуживания только пока занят, а он занят 55 % времени.

8. Проверка по Литтлу: $\overline{k} = \frac{6}{11}$, $t_{\text{сист}} = \frac{z}{A} = \frac{\frac{6}{11}}{\frac{2}{11}} = 3$ мин — время обслуживания принятой заявки равно среднему времени обслуживания, очереди нет.

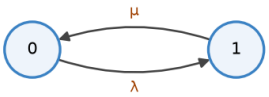


Рис. 2. Граф задачи 1.

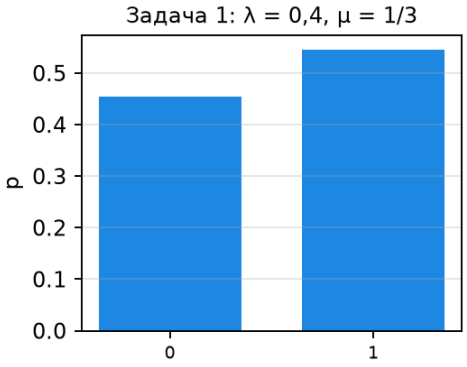


Рис. 3. Вероятности задачи 1 (как их показывает вкладка).

Программа при вводе $\lambda = 0.4$, $\mu = 0.3333$, $n = 1$, $m = 0$, без флага выдаёт те же числа: $p = (0.454545, 0.545455)$, $P_{\text{отк}} = 0.545455$, $A = 0.181818$, $t_{\text{сист}} = 3$.

4.3 Задача 3: сортировочная горка с неограниченной очередью

Условие. Простейший поток составов $\lambda = 2$ состава/час; время роспуска показательное, $t_{\text{обсл}} = 20$ мин. Одноканальная СМО с неограниченной очередью. Найти финальные вероятности, среднее число составов, связанных с горкой, длину очереди, времена пребывания.

Решение.

1. Единицы: $t_{\text{обсл}} = \frac{1}{3}$ ч, $\mu = 3$ состава/ч. $\rho = \frac{\lambda}{\mu} = \frac{2}{3} < 1$ — стационарный режим есть.
2. Состояния: $k = 0, 1, 2, \ldots$ — число составов, связанных с горкой (один на роспуске, остальные ждут).
3. Переходы: $k \to k+1$ с интенсивностью $\lambda = 2$, $k \to k-1$ с интенсивностью $\mu = 3$ (канал один, поэтому μ не растёт с k).
4. Сечение между k и $k+1$: $\lambda p_k = \mu p_{k+1}$, поэтому $p_{k+1} = \rho p_k$ и $p_k = \rho^k p_0$.
5. Нормировка: $p_0(1 + \rho + \rho^2 + \ldots) = \frac{p_0}{1-\rho} = 1$, откуда $p_0 = 1 - \rho = \frac{1}{3}$ и

$$p_k = \frac{1}{3} \cdot \left(\frac{2}{3}\right)^k : \quad p_1 = \frac{2}{9}, \quad p_2 = \frac{4}{27}, \quad p_3 = \frac{8}{81}, \ldots$$

6. Среднее число составов в системе:

$$z = \sum_{k=0}^{\infty} k p_k = \frac{\rho}{1-\rho} = \frac{\frac{2}{3}}{\frac{1}{3}} = 2.$$

7. Среднее число в очереди (из z вычитаем в среднем обслуживаемый состав: $1 - p_0 = \rho$):

$$L_{\text{оч}} = z - \rho = \frac{\rho^2}{1-\rho} = \frac{\frac{4}{9}}{\frac{1}{3}} = \frac{4}{3} \approx 1.33.$$

8. Формула Литтла ($P_{\text{отк}} = 0$, $\lambda_{\text{прин}} = \lambda$): $t_{\text{сист}} = \frac{z}{\lambda} = \frac{2}{2} = 1$ ч, $t_{\text{оч}} = \frac{L_{\text{оч}}}{\lambda} = \frac{\frac{4}{3}}{2} = \frac{2}{3}$ ч $= 40$ мин.
9. Проверка: $t_{\text{оч}} + t_{\text{обсл}} = 40 + 20 = 60$ мин $= t_{\text{сист}}$.

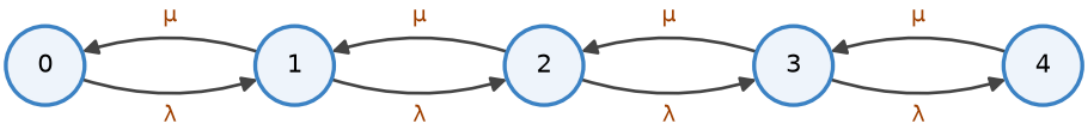


Рис. 4. Граф задачи 3 (показаны первые пять состояний, дальше цепочка продолжается).

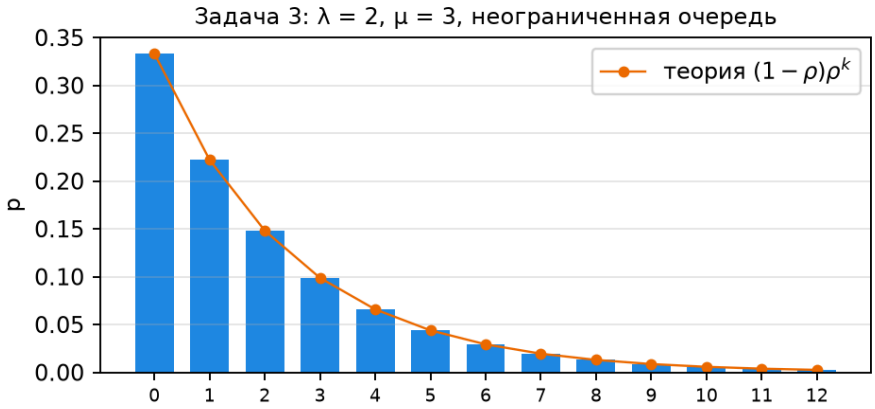


Рис. 5. Вероятности задачи 3: столбцы — результат решателя на 201 состоянии, точки — формула $(1-\rho)\rho^k$. Они совпадают.

Программа при $\lambda = 2$, $\mu = 3$, $n = 1$, флаге «unbounded queue» выдаёт $p_0 = 0.333333$, $z = 2.000000$, $L_{\text{оч}} = 1.333333$, $t_{\text{сист}} = 1.000000$, $t_{\text{оч}} = 0.666667$. Это те же числа в пределах 10^{-12}.

> Для остальных задач этого семейства (АЗС с очередью, двухканальная система с отказами и доходом, ограниченная очередь $m = 3$ и $m = 4$) используется та же вкладка, меняются только числа. Единицы вводятся такими, как в условии: если время дано в минутах, то и λ нужно ввести в вызовах в минуту.

5 Станки и оператор

Условие. Оператор обслуживает n станков. Каждый станок отказывает с интенсивностью λ (пока он исправен). Наладка — показательная с параметром $\mu = \frac{1}{t_{\text{обсл}}}$, оператор один. Станки, которые застали оператора занятым, ждут в очереди. Состояния — число неисправных станков. Найти финальные вероятности, A — среднее число станков, ремонтируемых оператором в единицу времени, w — среднее число неисправных станков, R — среднее число станков в очереди на наладку, $P_{\text{зан}}$ — вероятность занятости оператора.

В программе значения по умолчанию: $n = 5$, $\lambda = 0.2$, $\mu = 1$ (условие общее, чисел нет, поэтому они выбраны так, чтобы все эффекты были видны).

5.1 Граф

Состояние $k = 0, 1, \ldots, n$ — число неисправных станков. Исправных станков $n - k$, и каждый из них ломается с интенсивностью λ, поэтому **интенсивность перехода вверх убывает**:

$$k \to k+1 : (n-k)\lambda, \quad k \to k-1 : \mu \quad \text{(оператор один, поэтому } \mu \text{ не зависит от } k \text{)}.$$

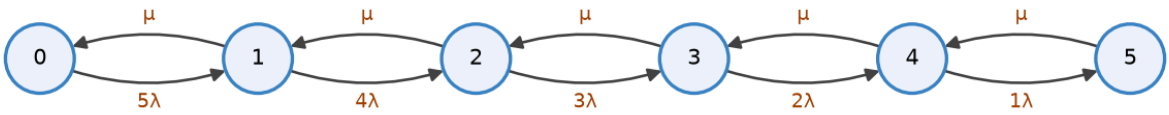


Рис. 6. Граф для $n = 5$: интенсивность вверх $(n-k)\lambda$, вниз μ.

Это отличие от задач с потоком заявок извне: источник заявок здесь **конечный**, и чем больше станков уже сломано, тем реже приходят новые заявки.

5.2 Решение

Сечение между k и $k+1$: $(n-k)\lambda p_k = \mu p_{k+1}$, откуда с $\rho = \frac{\lambda}{\mu}$

$$p_{k+1} = (n-k)\rho\, p_k, \quad p_k = \frac{n!}{(n-k)!}\,\rho^k\, p_0.$$

Для $n = 5$, $\rho = 0.2$ неотнормированные веса $u_k = \frac{n!}{(n-k)!} \cdot \rho^k$:

k	0	1	2	3	4	5
u_k	1	1	0.8	0.48	0.192	0.0384

Сумма весов 3.5104. Значит $p_0 = \frac{1}{3.5104} = 0.284868$ и $p_k = u_k p_0$:

k	0	1	2	3	4	5
p_k	0.284868	0.284868	0.227894	0.136737	0.054695	0.010939

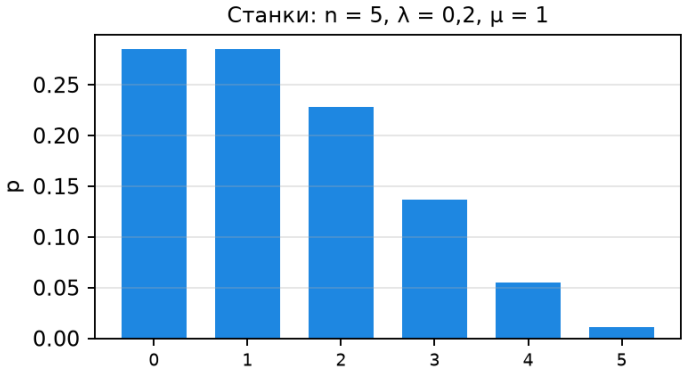


Рис. 7. Вероятности для $n = 5$, $\lambda = 0.2$, $\mu = 1$.

Характеристики:

- $P_{\text{зан}} = 1 - p_0 = 0.715132$ — оператор занят, когда есть неисправные станки.
- $A = \mu P_{\text{зан}} = 0.715132$ станка в единицу времени (оператор завершает μ наладок в единицу времени, пока занят).
- $w = \sum k p_k = 1.424339$.
- $R = \sum_{k \geq 1} (k-1) p_k = w - P_{\text{зан}} = 1.424339 - 0.715132 = 0.709207$ (из w вычитается тот станок, который сейчас налаживается).
- **Проверка.** Исправных станков в среднем $n - w = 3.575661$, и каждый ломается с интенсивностью λ, значит поток поломок равен $\lambda(n - w) = 0.2 \cdot 3.575661 = 0.715132 = A$. Совпадает: сколько ломается, столько и чинится.

6 Одноканальная СМО с отказами канала

Условие (задача 10). Одноканальная СМО с очередью, ограниченной $m = 2$ местами. Работающий канал может выходить из строя. Заявка, которая обслуживалась в момент отказа, становится в очередь, если там есть место, иначе покидает СМО необслуженной. Интенсивности: потока заявок $\lambda = 2$, обслуживания $\mu = 1$, отказов канала $Z = 0.5$, восстановлений $R = 1$. Найти состояния, финальные вероятности, относительную и абсолютную пропускные способности, среднее время пребывания заявки в системе.

> В условии не сказано, может ли канал выйти из строя, когда простаивает. Принято, что **нет**: отказывает только работающий (занятый) канал, потому что в следующей задаче сборника («канал может выходить из строя и в неработающем состоянии с интенсивностью $\gamma' < \gamma$») это допущение явно снимается. Если нужно иное, граф в `tasks/breakdown.rs` меняется добавлением перехода из состояния `up 0`.

6.1 Состояния

Состояние задаётся двумя величинами: исправен ли канал и сколько заявок в системе.

- `up k`, $k = 0, \ldots, m+1$: канал исправен; $k = 0$ — канал простаивает, $k \geq 1$ — одна заявка обслуживается и $k - 1$ ждут;
- `down k`, $k = 1, \ldots, m$: канал ремонтируется; обслуживаемой заявки нет, все k заявок стоят в очереди (мест m, поэтому $k \leq m$).

Для $m = 2$ получается шесть состояний: `up 0`, `up 1`, `up 2`, `up 3`, `down 1`, `down 2`. Состояния `down 0` нет: отказ происходит только при занятом канале, и заявка, которая обслуживалась, остаётся в системе (если есть место). Исключение $m = 0$: заявка, прервавшая обслуживание, теряется, и канал ремонтируется с пустой очередью, поэтому там состояние `down 0` существует и программа его создаёт.

6.2 Переходы

Из	В	Интенсивность и причина
`up k`, $k \leq m$	`up k+1`	λ — пришла заявка (при $k = m+1$ очередь полна, отказ, перехода нет)
`up k`, $k \geq 1$	`up k-1`	μ — закончилось обслуживание
`up k`, $k \geq 1$	`down min(k, m)`	Z — канал отказал; обслуживаемая заявка уходит в очередь (если $k \leq m$), иначе теряется (при $k = m+1$)
`down k`, $k < m$	`down k+1`	λ — пришла заявка (ремонт идёт, очередь пополняется)
`down k`	`up k`	R — канал отремонтирован

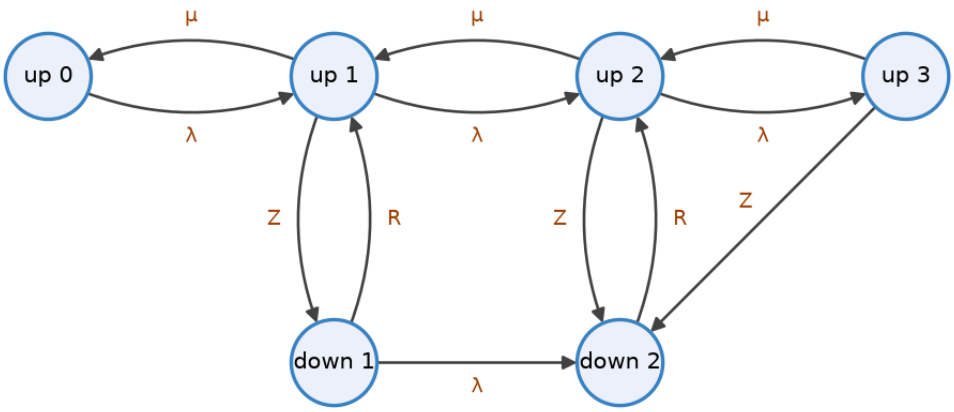


Рис. 8. Граф при $m = 2$. Из `up 3` отказ ведёт в `down 2`, потому что очередь уже полна и обслуживаемая заявка теряется.

6.3 Решение

Обозначим $a_k = p(\texttt{up}\quad k)$ и $d_k = p(\texttt{down}\quad k)$. Суммарные интенсивности выхода: из `up 0` это $\lambda = 2$; из `up 1`, `up 2` это $\lambda + \mu + Z = 3.5$; из `up 3` это $\mu + Z = 1.5$ (поступления блокированы); из `down 1` это $\lambda + R = 3$; из `down 2` это $R = 1$. Уравнения баланса (выход = вход):

$$
\begin{aligned}
\text{up } 0: &\quad 2a_0 = 1 \cdot a_1,\\
\text{up } 1: &\quad 3.5a_1 = 2a_0 + 1 \cdot a_2 + 1 \cdot d_1,\\
\text{up } 2: &\quad 3.5a_2 = 2a_1 + 1 \cdot a_3 + 1 \cdot d_2,\\
\text{up } 3: &\quad 1.5a_3 = 2a_2,\\
\text{down } 1: &\quad 3d_1 = 0.5a_1,\\
\text{down } 2: &\quad 1 \cdot d_2 = 2d_1 + 0.5a_2 + 0.5a_3.
\end{aligned}
$$

Решаем подстановкой, считая $a_0 = x$:

1. из `up 0`: $a_1 = 2x$;
2. из `down 1`: $d_1 = 0.5 \cdot 2\frac{x}{3} = \frac{x}{3}$;
3. из `up 1`: $a_2 = 3.5 \cdot 2x - 2x - \frac{x}{3} = 14\frac{x}{3}$;
4. из `up 3`: $a_3 = 2\frac{a_2}{1.5} = 56\frac{x}{9}$;
5. из `up 2`: $d_2 = 3.5a_2 - 2a_1 - a_3 = 49\frac{x}{3} - 4x - 56\frac{x}{9} = 55\frac{x}{9}$;
6. нормировка: $x\big(1 + 2 + \frac{14}{3} + \frac{56}{9} + \frac{1}{3} + \frac{55}{9}\big) = x \cdot \frac{183}{9} = 1$, откуда $x = \frac{9}{183}$.

Итого $(a_0, a_1, a_2, a_3, d_1, d_2) = \frac{9,18,42,56,3,55}{183} = (0.0492, 0.0984, 0.2295, 0.3060, 0.0164, 0.3005)$. Последнее уравнение (`down 2`) в решении не использовалось, и оно выполняется: $55 = 2 \cdot 3 + 0.5 \cdot 42 + 0.5 \cdot 56 = 6 + 21 + 28$. Это проверка.

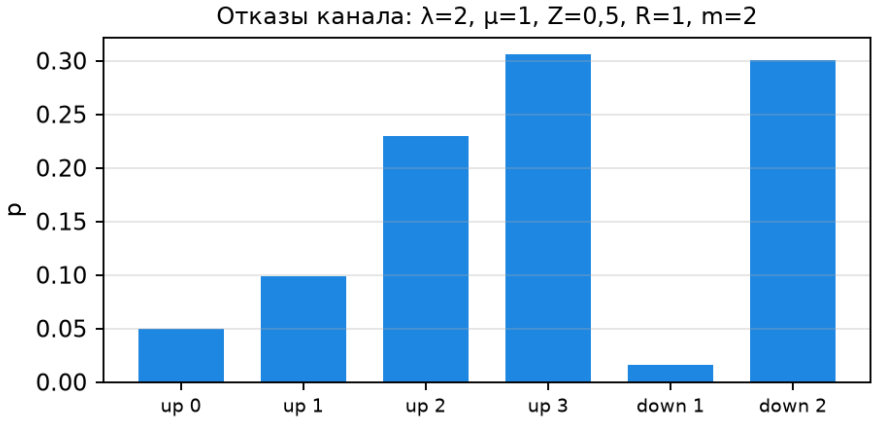


Рис. 9. Вероятности, найденные программой; совпадают с ручным решением.

6.4 Характеристики

- Вероятность того, что канал обслуживает: $\sum_{k\geq 1} a_k = \frac{18+42+56}{183} = \frac{116}{183}$.
- $\boldsymbol{A = \mu \cdot \frac{116}{183} = 0.633880}$ заявки в единицу времени (обслуживание завершается только при занятом исправном канале).
- $\boldsymbol{Q = \frac{A}{\lambda} = \frac{58}{183} = 0.316940}$ — доля заявок, которые в итоге обслужены. Она учитывает и отказы при полной очереди, и заявки, потерянные при отказе канала.
- Доля времени ремонта $d_1 + d_2 = \frac{58}{183} = 0.316940$. Это согласуется с балансом отказов и ремонтов: канал выходит из строя с интенсивностью Z только пока занят, поэтому $d_1 + d_2 = \big(\frac{Z}{R}\big) \cdot \frac{116}{183}$.
- Среднее число заявок в системе $z = \frac{18+2\cdot 42+3\cdot 56+3+2\cdot 55}{183} = \frac{383}{183} = 2.092896$.
- Заявка принимается в систему, если очередь не полна: блокирующие состояния `up 3` и `down 2` (m мест полны), их вероятность $\frac{111}{183}$. Интенсивность принятого потока $\lambda_{\text{прин}} = 2 \cdot \big(1 - \frac{111}{183}\big) = \frac{144}{183}$.
- $\boldsymbol{t_{\text{сист}} = \frac{z}{\lambda_{\text{прин}}} = \frac{383}{144} = 2.659722}$ (в единицах времени условия). Формула Литтла применима ко всем принятым заявкам, в том числе к тем, что потом потеряны при отказе канала: они провели в системе своё время.

7 Приоритет с вытеснением

Условие. Двухканальная СМО с отказами. Два простейших потока: I (интенсивность λ_1) и II (λ_2). Заявка I имеет приоритет: если она приходит, когда все каналы заняты и хотя бы один обслуживает заявку II, она вытесняет заявку II (та покидает СМО необслуженной) и занимает её место. Если оба канала заняты заявками I, заявка I получает отказ. Заявка II получает отказ, если заняты оба канала (безразлично какими заявками). Время обслуживания показательное с параметрами μ_1 и μ_2. Найти вероятности отказа для заявок обоих типов, среднее число обслуженных заявок каждого типа в единицу времени, среднее число занятых каналов.

В программе по умолчанию $\lambda_1 = 1$, $\lambda_2 = 2$, $\mu_1 = \mu_2 = 1$.

7.1 Состояния и переходы

Состояние (r, j): r заявок типа I и j типа II в системе, $r + j \le 2$ (очереди нет, все заявки в обслуживании). Шесть состояний: $(0,0)$, $(0,1)$, $(0,2)$, $(1,0)$, $(1,1)$, $(2,0)$.

Событие	**Переход**	**Интенсивность, условие**
пришла заявка I	$(r,j) \to (r+1,j)$	λ_1, если $r + j < 2$ (есть свободный канал)
пришла заявка I, все заняты	$(r,j) \to (r+1,j-1)$	λ_1, если $r + j = 2$ и $j > 0$ — **вытеснение** II
пришла заявка I, все заняты I	нет перехода	отказ; $r = 2$
пришла заявка II	$(r,j) \to (r,j+1)$	λ_2, если $r + j < 2$; иначе отказ
закончилась I	$(r,j) \to (r-1,j)$	$r\mu_1$
закончилась II	$(r,j) \to (r,j-1)$	$j\mu_2$

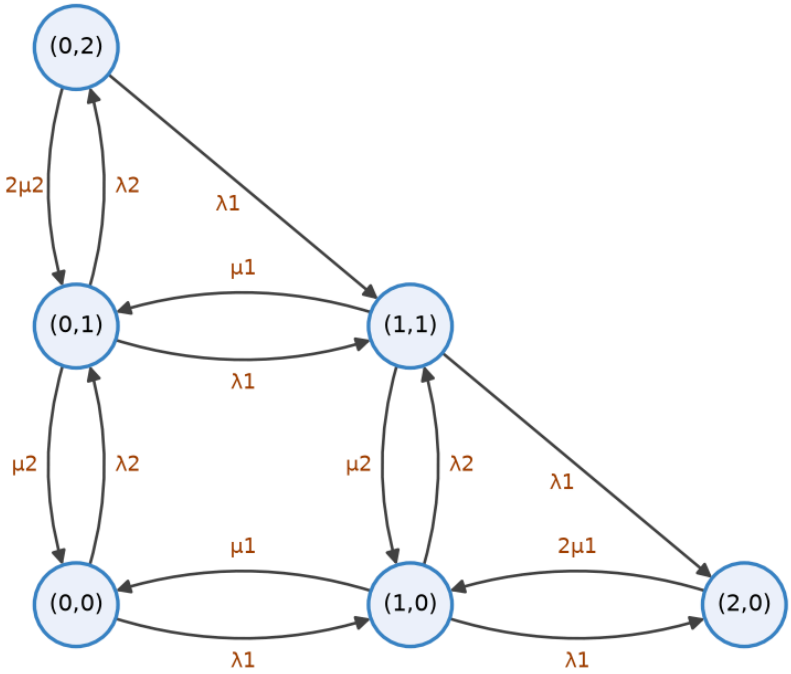


Рис. 10. Граф состояний. Диагональные стрелки — вытеснение заявки II заявкой I.

Особенность: у вытеснения нет обратного перехода по диагонали, поэтому две диагональные стрелки односторонние.

7.2 Решение

Пусть $a = p(0,0), b = p(0,1), c = p(0,2), d = p(1,0), e = p(1,1), f = p(2,0)$. При $\lambda_1 = 1, \lambda_2 = 2, \mu_1 = \mu_2 = 1$ уравнения баланса:

$$
\begin{aligned}
(0,2):&\quad (\lambda_1+2\mu_2)c=\lambda_2 b && \Longrightarrow 3c=2b,\\
(2,0):&\quad 2\mu_1 f=\lambda_1 d+\lambda_1 e && \Longrightarrow 2f=d+e,\\
(1,1):&\quad (\lambda_1+\mu_1+\mu_2)e=\lambda_1 b+\lambda_2 d+\lambda_1 c && \Longrightarrow 3e=b+2d+c,\\
(0,0):&\quad (\lambda_1+\lambda_2)a=\mu_1 d+\mu_2 b && \Longrightarrow 3a=d+b,\\
(1,0):&\quad (\lambda_1+\lambda_2+\mu_1)d=\lambda_1 a+2\mu_1 f+\mu_2 e && \Longrightarrow 4d=a+2f+e,\\
(0,1):&\quad (\lambda_1+\lambda_2+\mu_2)b=\lambda_2 a+2\mu_2 c+\mu_1 e && \Longrightarrow 4b=2a+2c+e.
\end{aligned}
$$

Считаем всё через b (шестое уравнение оставляем на проверку):

1. $c=2\frac{b}{3}$;
2. e из третьего: $3e=b+2d+2\frac{b}{3}=5\frac{b}{3}+2d$;
3. $a=\frac{d+b}{3}$ из четвёртого;
4. пятое: $4d=\frac{d+b}{3}+(d+e)+e$, то есть $4d=\frac{d+b}{3}+d+2e$, откуда $8d=b+6e$. Подставляя e из шага 2: $8d=b+10\frac{b}{3}+4d$, значит $d=13\frac{b}{12}$;
5. $e=\frac{5\frac{b}{3}+13\frac{b}{6}}{3}=23\frac{b}{18}$; $a=25\frac{b}{36}$; $f=\frac{d+e}{2}=85\frac{b}{72}$;
6. нормировка: $b\big(1+\frac{25}{36}+\frac{2}{3}+\frac{13}{12}+\frac{23}{18}+\frac{85}{72}\big)=425\frac{b}{72}=1$, откуда $b=\frac{72}{425}$.

Итого $(a,b,c,d,e,f)=\frac{50,72,48,78,92,85}{425}=(0.1176,0.1694,0.1129,0.1835,0.2165,0.2000)$. Проверка шестым уравнением: $4\cdot 72=288=2\cdot 50+2\cdot 48+92=100+96+92$.

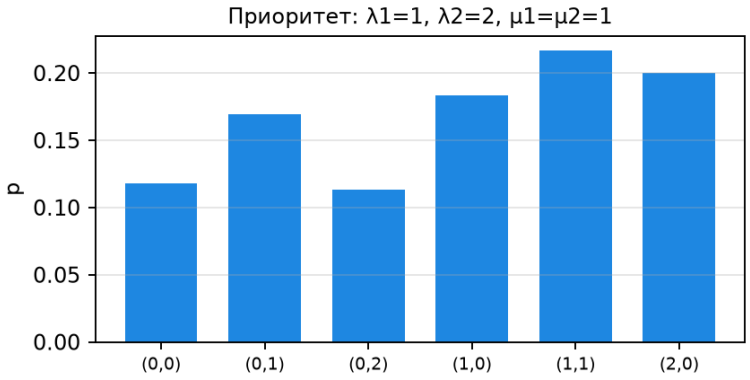


Рис. 11. Вероятности, найденные программой; совпадают с ручным решением.

7.3 Характеристики

- **Отказ для заявки I.** Заявка I отказывается, когда оба канала заняты I: $P_I=p(2,0)=\frac{85}{425}=0.2$. Вытеснение — не отказ для I.
- **Отказ для заявки II при поступлении.** Заявка II отказывается, когда заняты оба канала: $p(0,2)+p(1,1)+p(2,0)=\frac{48+92+85}{425}=\frac{225}{425}=0.5294$.
- **Заявка II не обслужена (отказ или вытеснение).** В отличие от I, заявку II могут прервать уже в обслуживании. Поэтому честная доля обслуженных II считается по завершениям обслуживания: $A_2=\mu_2\sum_{r,j} j\,p(r,j)=\frac{72+2\cdot 48+92}{425}=\frac{260}{425}=0.6118$. Тогда необслуженных $1-\frac{A_2}{\lambda_2}=1-0.3059=0.6941$.
- $\boldsymbol{A_1=\lambda_1(1-P_I)=0.8}$ обслуженных заявок I в единицу времени (они не прерываются, поэтому всё принятое обслуживается). Проверка: $\mu_1\sum rp=\frac{78+92+2\cdot 85}{425}=\frac{340}{425}=0.8$.
- **Среднее число занятых каналов** $\sum(r+j)p=\frac{72+96+78+184+170}{425}=\frac{600}{425}=1.4118$. Проверка: $\frac{A_1}{\mu_1}+\frac{A_2}{\mu_2}=0.8+0.6118=1.4118$ (нагрузка каждого типа равна числу его занятых каналов).

8 Метод фаз: обслуживание по закону Эрланга

Условие (задача про ЭВМ). Одноканальная СМО, на вход поступает простейший поток со средним интервалом между заявками 10 мин. Время обслуживания распределено по закону Эрланга 3-го порядка с математическим ожиданием 8 мин. Определить среднее число z заявок в СМО и в очереди $L_{\text{оч}}$, а также средние времена пребывания заявки в системе $t_{\text{сист}}$ и в очереди $t_{\text{оч}}$.

В программе по умолчанию заданы именно эти числа: $\lambda = 0.1$ заявок/мин, три фазы, $\nu_1 = \nu_2 = \nu_3 = 0.375$, неограниченная очередь.

8.1 Идея метода

Время по закону Эрланга порядка k можно представить как сумму k независимых показательных времён («фаз»). Если заявка обслуживается по фазам, то **положение внутри обслуживания** тоже можно сделать частью состояния. Тогда в каждый момент до конца текущей фазы остаётся показательное время: свойство отсутствия памяти (раздел 2.3) возвращается, и граф с постоянными интенсивностями снова подходит.

- **Эрланг порядка k с математическим ожиданием t:** k фаз с одной и той же интенсивностью $\nu = \frac{k}{t}$. В задаче: $\nu = \frac{3}{8} = 0.375$.
- **Обобщённый Эрланг:** интенсивности фаз $\nu_1, \ldots, \nu_k$ различны. Среднее время обслуживания $t = \sum \frac{1}{\nu_i}$, дисперсия $\sigma^2 = \sum \frac{1}{\nu_i^2}$.

Программа сразу принимает разные ν_i (поля $\nu_1 \ldots \nu_5$, используются первые k), поэтому одинаковые интенсивности это частный случай.

> Заявка проходит фазы подряд, и только после последней фазы канал освобождается. Новая заявка начинает с первой фазы. Фаза не имеет физического смысла: это приём, который превращает немарковскую задачу в марковскую цепь.

8.2 Состояния и переходы

Состояние 0: система пуста. Состояние (j, i): в системе j заявок, обслуживаемая находится в фазе i. Число состояний $1 + k(m+1)$ при m местах в очереди.

Событие	Переход	Интенсивность, условие
пришла заявка в пустую систему	$0 \to (1,1)$	λ
пришла заявка, канал занят	$(j,i) \to (j+1,i)$	λ, если $j \le m$ (в очереди есть место); фаза не меняется
закончилась фаза i	$(j,i) \to (j,i+1)$	ν_i, если $i < k$
закончилась последняя фаза, есть очередь	$(j,k) \to (j-1,1)$	ν_k, $j > 1$
закончилась последняя фаза, очереди нет	$(1,k) \to 0$	ν_k

Интенсивность перехода в следующую фазу равна ν_i именно **покидаемой** фазы. Здесь легко ошибиться: переход $(2,2) \to (2,3)$ идёт с ν_2, а не с ν_3.

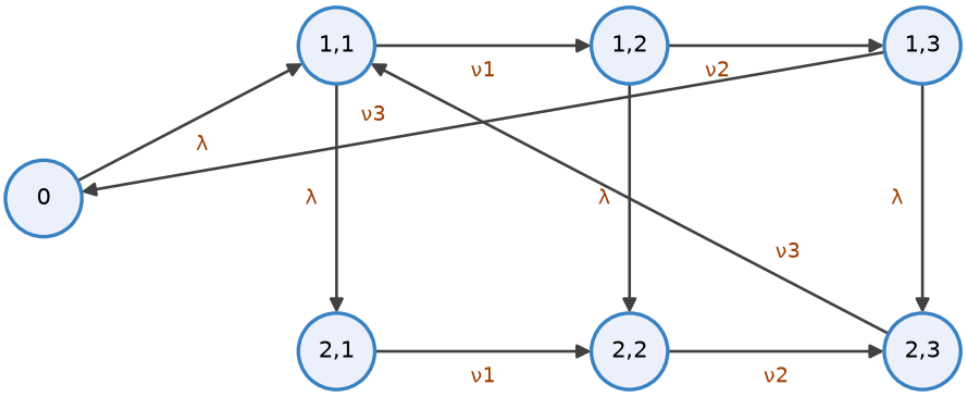


Рис. 12. Граф состояний для $k = 3$, $m = 1$ (7 состояний). Верхняя строка: одна заявка, нижняя: две заявки.

8.3 Задача без очереди и «нечувствительность»

Пусть $m = 0$, $k = 2$. Состояний три: 0, $(1,1)$, $(1,2)$; обозначим вероятности p_0, p_{11}, p_{12}. Уравнения баланса:

$$\begin{aligned} 0: &\quad \lambda p_0 = \nu_2 p_{12}, \\ (1,1): &\quad \nu_1 p_{11} = \lambda p_0, \\ (1,2): &\quad \nu_2 p_{12} = \nu_1 p_{11}. \end{aligned}$$

Отсюда $p_{11} = \lambda \frac{p_0}{\nu_1}$, $p_{12} = \lambda \frac{p_0}{\nu_2}$, и из нормировки $p_0\left(1 + \frac{\lambda}{\nu_1} + \frac{\lambda}{\nu_2}\right) = 1$. Обозначим $a = \lambda\left(\frac{1}{\nu_1} + \frac{1}{\nu_2}\right) = \lambda t_{\text{обсл}}$. Тогда $p_0 = \frac{1}{1+a}$ и

$$P_{\text{отк}} = p_{11} + p_{12} = \frac{a}{1+a}.$$

Результат зависит только от среднего времени обслуживания, а не от разбиения на фазы: система с отказами **нечувствительна** к закону времени обслуживания (формула Эрланга). Этим пользуются в задачах с фиксированным временем обслуживания (раздел 10). Тест `phases::loss_system_is_insensitive_to_the_service_law` проверяет равенство для $\nu_1 = 2$, $\nu_2 = 0.5$.

8.4 Характеристики из вектора p

Все характеристики функция задачи выводит из $p_{j,i}$, суммируя по фазам:

- $z = \sum_{j,i} j\, p_{j,i}$, $L_{\text{оч}} = \sum_{j,i} (j-1) p_{j,i}$;
- $P_{\text{отк}} = \sum_i p_{m+1,i}$ (все места заняты; для неограниченной очереди $P_{\text{отк}} = 0$);
- $A = \lambda(1 - P_{\text{отк}})$, $Q = 1 - P_{\text{отк}}$;
- $P_{\text{занят}} = 1 - p_0$;
- $t_{\text{сист}} = \frac{z}{A}$, $t_{\text{оч}} = \frac{L_{\text{оч}}}{A}$ (формула Литтла по принятому потоку);
- среднее время обслуживания $\sum \frac{1}{\nu_i}$.

8.5 Решение задачи про ЭВМ

Для неограниченной очереди граф бесконечен, и программа обрезает его на 100 заявках (301 состояние при $k = 3$). Для ручной проверки используем формулу Поллачека–Хинчина для одноканальной системы с произвольным временем обслуживания (математическое ожидание t, дисперсия σ^2, $\rho = \lambda t$):

$$L_{\text{оч}} = \frac{\lambda^2 \sigma^2 + \rho^2}{2(1-\rho)}.$$

В задаче $\lambda = 0.1$, $t = 8$ мин, $\rho = 0.8 < 1$, $\sigma^2 = \frac{3}{0.375^2} = 21.333$ мин^2.

Величина	Формула	Значение
$L_{\text{оч}}$	$\frac{0.01 \cdot 21.333 + 0.64}{2 \cdot 0.2}$	2.1333
z	$L_{\text{оч}} + \rho$	2.9333
$t_{\text{оч}}$	$\frac{L_{\text{оч}}}{\lambda}$	21.33 мин
$t_{\text{сист}}$	$\frac{z}{\lambda}$	29.33 мин

)

Фазовая модель в программе даёт то же $L_{\text{оч}}$ с точностью 10^{-4} (тест `phases::computer_with_erlang_3_service_matches_pollaczek_khinchine`); для разных $\nu = (1, 2, 4)$ при $\lambda = 0.3$ совпадение до 10^{-6}.

При тех же λ и t, но показательном времени обслуживания ($k = 1$) было бы $L_{\text{оч}} = \frac{\rho^2}{1-\rho} = 3.2$. Меньший разброс времени у Эрланга-3 сокращает очередь на треть.

9 Магазин: три СМО подряд

Условие. Покупатели приходят простейшим потоком $\lambda = 45$ чел/ч и проходят три фазы: 1) выбор товара (4 точки, среднее время 5 мин); 2) касса (одна, среднее время 1 мин); 3) контроль (3 продавца, время в условии не указано: в программе это параметр t_3, по умолчанию 1 мин). Найти показатели магазина и ответить, в каком звене нужно улучшить обслуживание, чтобы сократить время покупателей.

9.1 Почему звенья считаются по отдельности

Граф всей системы был бы трёхмерным, (k_1, k_2, k_3), и очень большим. Но по теореме Бёрке **на выходе стационарной системы $M/M/n$ с неограниченной очередью поток тоже простейший с той же интенсивностью λ**. Значит, каждое следующее звено получает простейший поток λ и решается само по себе. Условие стационарности записывается для каждого звена: $\rho_i = \frac{\lambda}{n_i \mu_i} < 1$.

Время в магазине равно сумме времён в звеньях, число покупателей в магазине равно сумме по звеньям (или λ на общее время, формула Литтла). Отказов нет: $P_{\text{отк}} = 0$, $Q = 1$, $A = \lambda$.

9.2 Формулы Эрланга С

Для звена $M/M/n$ с неограниченной очередью: $\alpha = \frac{\lambda}{\mu}$, $\rho = \frac{\alpha}{n} < 1$. Граф тот же, что в разделе 4: $p_k = \alpha^k \frac{p_0}{k!}$ при $k \le n$ и $p_{n+s} = p_n \rho^s$ в очереди. Нормировка даёт

$$p_0 = \left[\sum_{k=0}^{n-1} \frac{\alpha^k}{k!} + \frac{\alpha^n}{n!(1-\rho)} \right]^{-1}.$$

Вероятность ожидания равна вероятности, что заняты все каналы: $P_{\text{ждёт}} = \left(\frac{\alpha^n}{n!(1-\rho)}\right) p_0$. Остальное:

$$L_{\text{оч}} = P_{\text{ждёт}} \cdot \frac{\rho}{1-\rho}, \quad t_{\text{оч}} = \frac{L_{\text{оч}}}{\lambda}, \quad t_{\text{сист}} = t_{\text{оч}} + \frac{1}{\mu}.$$

Для одного канала ($n = 1$): $P_{\text{ждёт}} = \rho$, $L_{\text{оч}} = \frac{\rho^2}{1-\rho}$.

Эти формулы выбраны вместо вкладки «Channels and queue», потому что при $\rho_1 = 0.9375$ обрезание очереди на 200 заявках оставило бы хвост порядка 10^{-6} (программа показала бы предупреждение), а формулы точны.

9.3 Решение

Интенсивность обслуживания на канал: $\mu_1 = \frac{60}{5} = 12$ чел/ч, $\mu_2 = \frac{60}{1} = 60$ чел/ч, $\mu_3 = 60$ чел/ч (при $t_3 = 1$ мин).

Звено 1 (выбор). $\alpha = \frac{45}{12} = 3.75$, $n = 4$, $\rho_1 = 0.9375$. Сумма $\sum_{k=0}^{3} \frac{\alpha^k}{k!} = 1 + 3.75 + 7.0313 + 8.7891 = 20.5703$, слагаемое $\frac{\alpha^4}{4!(1-\rho)} = \frac{8.2397}{0.0625} = 131.836$. $P_{\text{ждёт}} = \frac{131.836}{20.5703 + 131.836} = 0.8650$. $L_{\text{оч}} = 0.8650 \cdot \frac{0.9375}{0.0625} = 12.975$, $t_{\text{оч}} = \frac{12.975}{45} = 0.2883$ ч $= 17.30$ мин, $t_{\text{сист}} = 17.30 + 5 = 22.30$ мин.

Звено 2 (касса). $\rho_2 = \frac{45}{60} = 0.75$, $L_{\text{оч}} = \frac{0.75^2}{0.25} = 2.25$, $t_{\text{оч}} = 3.0$ мин, $t_{\text{сист}} = 4.0$ мин.

Звено 3 (контроль, $n = 3$). $\alpha = 0.75$, $\rho_3 = 0.25$, $P_{\text{ждёт}} = 0.0441$, $L_{\text{оч}} = 0.0147$, $t_{\text{оч}} = 0.02$ мин, $t_{\text{сист}} = 1.02$ мин.

Звено	ρ	$L_{\text{оч}}$	$t_{\text{оч}}$, мин	$t_{\text{сист}}$, мин
1. Выбор ($n = 4$, 5 мин)	0.9375	12.98	17.30	22.30
2. Касса ($n = 1$, 1 мин)	0.75	2.25	3.00	4.00
3. Контроль ($n = 3$, 1 мин)	0.25	0.015	0.02	1.02

Итого: время в магазине 27.32 мин, в магазине в среднем $45 \cdot \frac{27.32}{60} = 20.49$ человека.

9.4 Ответ на дополнительный вопрос

Узкое место: **выбор товара**. Здесь самая высокая загрузка (0.9375), самая длинная очередь (13 человек), и 17 из 27 минут уходит на ожидание. Звенья связаны формулой Литтла $L_{\text{оч}} = \lambda t_{\text{оч}}$ при одном и том же λ, поэтому сравнение по длине очереди и по времени ожидания даёт один результат.

Что даёт улучшение (результаты программы при $t_3 = 1$ мин):

Изменение	**Время в магазине, мин**
без изменений	27.3
кассу ускорить вдвое ($t_2 = 0.5$ мин)	24.1
пятая точка выбора ($n_1 = 5$)	11.9

Пятая точка сокращает время более чем вдвое: ρ_1 падает с 0.9375 до 0.75, ожидание в первом звене с 17.3 до 1.8 мин. Ускорение кассы сокращает время лишь на 3 минуты, потому что очередь в кассе и так короткая.

10 Остальные немарковские задачи (не реализованы)

Эти задачи разобраны до метода решения, но в программе их нет.

- **Задача 2: фиксированное время обслуживания, отказы.** Для системы с отказами результат зависит только от среднего времени обслуживания (раздел 8.3), поэтому для одного канала $P_{\text{отк}} = \frac{\alpha}{1+\alpha}$. Если время обслуживания равно среднему интервалу между заявками, то $\alpha = 1$ и $Q = \frac{1}{1+\alpha} = 0.5$. В программе это вкладка «Phase-type service» с $m = 0$.
- **Столовая (смесь времён обслуживания).** Одноканальная СМО, где время обслуживания случайно (одно блюдо или два). Решается формулой Поллачека–Хинчина (раздел 8.5), где t и σ^2 считаются как математическое ожидание и дисперсия смеси; стационарность: $\lambda t < 1$.
- **Две фазы с возвратом (сеть с обратной связью).** Если вероятность успешного прохода равна p, то каждая заявка проходит СМО 1 в среднем $1/p$ раз, и входной поток каждого звена равен λ/p (из уравнения $\lambda_1 = \lambda + (1-p)\lambda_1$). Условия стационарности: $\lambda(1/Y_1 + (1/p-1)/Y_{12}) < n_1$ и $\lambda(1/Y_2 + (1/p-1)/Y_{22}) < n_2$ (первый проход с интенсивностью Y, повторные с Y_{12} и Y_{22}). Простейшие потоки в звеньях задача предписывает считать допущением.
- **Задача с «обобщённым Эрлангом» и $R = 2$.** В docx условие обрывается («параметрами k, X_2»), определение не дано. Вкладка «Phase-type service» с $m = 2$ покрывает типичную постановку, но параметры нужно уточнить.
- **АБД** (многоканальная СМО с неограниченной очередью) решается вкладкой «Channels and queue», $n = 6$, $\mu = 60$.

11 Тесты

24 теста запускаются командой `cargo test -p lab4`.

Тест	Что проверяет
`chain::two_state_chain_matches_the_hand_solution`	Решатель на задаче 1 даёт $\left(\frac{5}{11}, \frac{6}{11}\right)$ с точностью 10^{-12}.
`chain::disconnected_chain_has_no_unique_solution`	Граф с изолированным состоянием не даёт решения (`None`).
`queue::task_1_single_channel_with_refusals`	p_0, $P_{\text{отк}}$, A задачи 1.
`queue::task_3_unbounded_queue_matches_the_formulas`	$p_0 = \frac{1}{3}$, $z = 2$, $L_{\text{оч}} = \frac{4}{3}$, $t_{\text{сист}} = 1$, $t_{\text{оч}} = \frac{2}{3}$.
`queue::overloaded_unbounded_queue_is_rejected`	При $\lambda \geq n\mu$ неограниченная очередь отклоняется.
`queue::two_channels_without_queue_match_erlang_b`	Два канала без очереди: $P_{\text{отк}}$ совпадает с формулой Эрланга.
`machines::single_machine_alternates_between_two_states`	Один станок: $p_0 = \frac{\mu}{\lambda+\mu}$.
`machines::repaired_per_unit_time_equals_broken_machines_flow`	$A = \lambda(n - w)$.
`breakdown::without_failures_it_is_the_plain_queue`	При $Z \to 0$ задача сводится к $M/M/1$ с m местами (проверка по вкладке «Channels and queue»).
`breakdown::all_cases_of_m_give_a_distribution`	Для $m = 0…4$ сумма p равна 1, число подписей равно числу вероятностей.
`breakdown::repair_time_share_matches_a_failure_repair_cycle`	Доля ремонта равна $\frac{Z}{R}$, умноженному на вероятность занятого канала.
`priority::type_i_alone_sees_an_erlang_loss_system`	При $\lambda_2 \to 0$ отказ для I равен формуле Эрланга для двух каналов.
`priority::type_i_served_flows_agree`	$A_1 = \mu_1 E[r]$.
`priority::busy_channels_balance_with_served_flows`	Занятые каналы равны $\frac{A_1}{\mu_1} + \frac{A_2}{\mu_2}$.
`phases::loss_system_is_insensitive_to_the_service_law`	Без очереди $P_{\text{отк}} = \frac{a}{1+a}$, $a = \lambda \sum \frac{1}{\nu_i}$, при разных ν_i.
`phases::one_phase_is_the_plain_single_channel_queue`	Одна фаза даёт те же A и z, что вкладка «Channels and queue».
`phases::computer_with_erlang_3_service_matches_pollaczek_khinchine`	Задача про ЭВМ: $L_{\text{оч}} = 2.1333$ по формуле Поллачека–Хинчина.
`phases::unequal_phase_rates_match_pollaczek_khinchine`	То же при разных $\nu_i = (1, 2, 4)$.
`phases::overloaded_unbounded_queue_is_rejected`	При $\lambda t_{\text{обсл}} \geq 1$ неограниченная очередь отклоняется.
`phases::states_are_a_distribution_for_every_phase_count`	Для $k = 1…5$ сумма p равна 1, число подписей равно числу вероятностей.
`shop::single_channel_matches_mm1_formulas`	Один канал: $L_{\text{оч}} = \frac{\rho^2}{1-\rho}$, $t = \frac{1}{\mu-\lambda}$.
`shop::multi_channel_matches_the_queue_tab`	Формулы Эрланга C совпадают со вкладкой «Channels and queue» (4 канала).

`shop::shop_totals_add_up_and_the_busiest_stage_is_the_bottleneck`	Итоговое время равно сумме по звеньям, формула Литтла для магазина, узкое место, $\rho_1 = 0.9375$, $\rho_2 = 0.75$.
`shop::overloaded_stage_is_rejected_with_its_number`	Звено с $\rho \geq 1$ даёт ошибку с номером звена.

Кроме тестов, числа из разделов 3–7 (все p и характеристики по умолчанию) сверены с независимой реализацией на Python (`numpy.linalg.lstsq`, система уравнений баланса записана отдельно от Rust-кода): максимальное расхождение $5 \cdot 10^{-13}$.

12 Допущения и ограничения

- **Единицы.** Программа не конвертирует единицы. Если в условии время дано в минутах, а поток в часах, нужно привести к одним единицам вручную до ввода. Результаты по времени ($t_{\text{сист}}$, $t_{\text{оч}}$) будут в тех же единицах.
- **Параметр μ.** В полях вводится μ, а не среднее время обслуживания; $\mu = \frac{1}{t_{\text{обсл}}}$.
- **Неограниченная очередь** заменяется очередью в 200 мест (раздел 3.4). Для ρ близкого к 1 нужно смотреть на красное предупреждение.
- **Задача про отказы канала** реализована в предположении, что канал отказывает только при обслуживании (раздел 6). Это допущение, а не следствие условия.
- **Приоритетная задача** считает для двух каналов (константа `CHANNELS` в коде); число каналов менять нужно вместе с проверками формул.
- **Метод фаз.** Не более 5 фаз; неограниченная очередь обрезается на 100 заявках (до 501 состояния), при заметной вероятности обрезанного хвоста показывается предупреждение.
- **Магазин.** Допущения: у выбора товара есть очередь (иначе это была бы система с отказами), t_3 в условии не указано и вводится параметром (по умолчанию 1 мин), время в окне задаётся в минутах, поток в чел/ч (здесь конвертация внутри задачи).
- **Что не реализовано.** Задача 2 (фиксированное время обслуживания, отказы), столовая (смесь времён обслуживания), две фазы с возвратом (сеть с обратной связью), задача с «обобщённым Эрлангом» и $R = 2$ (в `docx` недописано условие), нетерпеливые заявки, «разогрев», взаимопомощь каналов, такси. Часть решается тем же `Chain` или формулами, см. раздел 10.
- **Граф состояний** в окне не рисуется (только столбцы и таблицы). Рисунки графов в этом документе построены отдельно.